



A
Project Report
on
Development of LLVM Backend for MRISC32 ISA using GlobalISel
submitted as partial fulfillment for the award of
BACHELOR OF TECHNOLOGY
DEGREE
SESSION 2025-26
in
Computer Science & Engineering (Artificial Intelligence & Machine Learning)
By
Shobhit Sinha (2200291530105)
Under the supervision of
Dr. Pratibha Singh
KIET Group of Institutions, Ghaziabad
Affiliated to
Dr. A.P.J. Abdul Kalam Technical University, Lucknow
(Formerly UPTU)
May, 2026

DECLARATION

I hereby declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Signature:

Name(s): Shobhit Sinha

Roll No.(s): 2200291530105

Date:



KIET
DEEMED TO BE
UNIVERSITY
DELHI - NCR, INDIA
(Under Section 3 of the UGC Act, 1956)

CERTIFICATE

This is to certify that Project Report entitled "Development of LLVM Backend for MRISC32 ISA using GlobalISel" which is submitted by the student in partial fulfillment of the requirement for the award of degree B. Tech. in the Department of Computer Science & Engineering (Artificial Intelligence & Machine Learning) of Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of the candidate's own work carried out by him under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Dr. Pratibha Singh
Associate Professor
CSE(AI&ML)

Dr. Rekha Kashyap
Dean
CSE(AI&ML)

Date:

ACKNOWLEDGEMENT

It gives me a great sense of pleasure to present the report of the B. Tech Project undertaken during B. Tech. Final Year. I owe special debt of gratitude to my supervisor Dr. Pratibha Singh, Department of Computer Science & Engineering (Artificial Intelligence & Machine Learning), KIET Group of Institutions, Ghaziabad, for her constant support and guidance throughout the course of my work. Her sincerity, thoroughness and perseverance have been a constant source of inspiration for us. It is only her cognizant efforts that my endeavors have seen the light of the day.

I also take the opportunity to acknowledge the contribution of Dr. Rekha Kashyap, Dean, Computer Science & Engineering (Artificial Intelligence & Machine Learning) Department, KIET Group of Institutions, Ghaziabad, for her full support and assistance during the development of the project. I also do not like to miss the opportunity to acknowledge the contribution of all the faculty members of the department for their kind assistance and cooperation during the development of my project. Last but not the least, I acknowledge my friends for their contribution in the completion of the project.

Date:

Signature(s):

Name(s): Shobhit Sinha

Roll No.: 2200291530105

ABSTRACT

This report presents the development and working of a LLVM backend for the MRISC32 Instruction Set Architecture (ISA), utilizing the GlobalISel Instruction Selection framework of LLVM compiler infrastructure. The MRISC32 ISA is an open-source RISC-style load-store vector architecture featuring 32 general-purpose registers of 32 bit width and 32-bit wide instructions encoded in 5 different formats.

This compiler backend implements the support to generate MRISC32 assembly or machine code from the LLVM Intermediate Representation (IR). The key components implemented internally to support the assembly and machine code generation includes register modeling using TableGen domain specific language (DSL), instruction definitions for all 5 instruction formats, and the complete GlobalISel pipeline consisting of IRTranslator, Legalizer, RegBankSelect, and InstructionSelect stages. Additionally, the calling convention of the MRISC32 ISA is correctly handled by passing the first eight function arguments through registers R1 to R8.

A comparison between the generated assembly from the LLVM backend and the existing GCC backend demonstrates semantic equivalence for the supported Base Module instructions. The current implementation establishes a functional foundation for MRISC32 within the LLVM ecosystem.

Future work includes extending support to optional ISA modules such as vector, floating-point, and packed operations, and leveraging the LLVM infrastructure to develop a native linker and standard library.

Keywords: LLVM, Compiler Backend, MRISC32, GlobalISel, Instruction Selection, TableGen, ELF

TABLE OF CONTENTS

DECLARATION	ii
CERTIFICATE	iii
ACKNOWLEDGEMENTS	iv
ABSTRACT	v
LIST OF FIGURES	vii
LIST OF ABBREVIATIONS	viii
CHAPTER 1 – INTRODUCTION	1
1.1 Introduction	1
1.2 Project Description	2
CHAPTER 2 – LITERATURE REVIEW	4
2.1 LLVM Compiler Infrastructure	4
2.2 MRISC32 ISA	4
2.3 Instruction Selection Frameworks	4
2.3.1 SelectionDAG	5
2.3.2 GlobalISel	5
CHAPTER 3 – PROPOSED METHODOLOGY	6
3.1 TableGen Domain Specific Language	6
3.2 Backend Design	6
3.2.1 Register Information	6
3.2.2 Instruction Information	6
3.3 GlobalISel Pipeline	7
3.3.1 IRTranslator	7
3.3.2 Legalizer	8
3.3.3 RegBankSelect	9
3.3.4 InstructionSelect	9
3.3.5 Calling Convention Handling	10
3.3.6 Function Prologue and Epilogue Generation	12
3.3.7 Assembly Generation	14
3.3.8 ELF Generation	14
CHAPTER 4 – RESULTS AND DISCUSSION	16
4.1 Test Function	16
4.2 GCC Backend Output	16
4.3 LLVM Backend Output	17
4.4 Comparison and Analysis	18
CHAPTER 5 – CONCLUSION AND FUTURE SCOPE	20
5.1 Conclusion	20
5.2 Future Scope	20
REFERENCES	22
RESEARCH PAPER	23
PROOF OF SUBMISSION	24
PLAGIARISM REPORT: AI SIMILARITY	25
PLAGIARISM REPORT: OVERALL SIMILARITY	27

LIST OF FIGURES

Figure No.	Description	Page No.
1.1	LLVM backed for MRISC32 ISA fits in the backend of the LLVM infrastructure along with backend for other architectures like x86, ARM, RISC-V, etc.	3
3.1	Tablegen register classes GPR and GPRDst in MRISC32RegisterInfo.td	6
3.2	Instruction format A defined as “FmtA” in tablegen file MRISC32InstrFormat.td	7
3.3	Two different formats of ADD instruction defined in MRISC32InstrInfo.td by inheriting the instruction formats B and C (FmtB and FmtC)	7
3.4	<i>MRISC32LegalizerInfo</i> class constructor containing logic to decide whether operation is natively handled or needs custom handling	8
3.5	Custom logic to handle 64 bit addition written in C++	9
3.6	Tablegen instruction to define pattern matching	10
3.7	<i>lowerFormalArguments()</i> is responsible for passing the first 8 function parameters to register R1 to R8	11
3.8	<i>lowerReturn()</i> is responsible for moving return value to register R1	11
3.9	<i>emitPrologue()</i> is responsible for generation prologue for every function	13
3.10	<i>emitEpilogue()</i> is responsible for generating epilogue at the end of every function	13
3.11	<i>MRISC32InstPrinter</i> class utilized for defining functions to print instructions and operands in proper format	14
3.12	<i>encodeInstruction()</i> in <i>MRISC32MCCodeEmitter.cpp</i> used to generate the correct opcode	15
4.1	Test program written in C consisting of sequence of arithmetic operations	16
4.2	MRISC32 assembly generated by the existing GCC backend	17
4.3	LLVM IR generated from clang using <i>-emit-llvm</i> flag	18
4.4	MRISC32 assembly generated from LLVM IR using LLVM backend for MRISC32	18

LIST OF ABBREVIATIONS

ISA: Instruction Set Architecture
LLVM: Low Level Virtual Machine
IR: Intermediate Representation
GlobalISel: Global Instruction Selection
SelectionDAG: Selection Directed Acyclic Graph
DSL: Domain Specific Language
MIR: Machine Intermediate Representation
gMIR: Generic Machine Intermediate Representation
ELF: Executable and Linkable Format
MachO: Mach Object
GCC: GNU Compiler Collection
RISC: Reduced Instruction Set Computing
GPR: General Purpose Register
MCPhysReg: Machine Code Physical Register
MLIR: Multi-Level Intermediate Representation
VM: Vector Module
PM: Packed Operation Module
FM: Floating-point Module
SM: Saturating and Halving Arithmetic Module

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Instruction Set Architecture (ISA) is the interface available to the programmer through which a processor is programmed. Since modern ISAs and processors are highly complex in nature, it is impractical to consistently write highly optimized programs for even trivial tasks directly in assembly or machine code [1]. Due to this limitation, the development of any modern ISA infrastructure is incomplete without the development of a compiler toolchain to support it. Modern compiler infrastructure frameworks like LLVM greatly reduce the time and effort required to create such a toolchain.

The process of compiler construction can be mainly broken down into 3 major parts. These are the compiler frontend, middlend and the backend [2].

Compiler frontend is the part of the compiler which is responsible for dealing with the syntax and the semantics of the programming language being compiled. This includes stages like Lexing, Parsing and Semantic Analysis. During Lexing, the source code is broken into a stream of tokens, from which an Abstract Syntax Tree (AST) is generated during Parsing using the grammar rules of the programming language being compiled.

Compiler middle-end is the part of the compiler where the target independent transformation or analysis of the code being compiled takes place. This stage is neither heavily dependent on the target architecture for which the source code is being compiled, nor on grammar rules of the programming language which is being compiled. This generally includes certain types of analysis and optimizations which help generate more efficient target code. Generally speaking, the compiler frontend emits an Intermediate Representation (IR) after performing steps like Parsing and Semantic Analysis. This IR is fed to the compiler middle-end where a series of optimization and transformations may take place on this IR. Some common optimization techniques which are performed at this stage are Dead Code Elimination (DCE), Common Subexpression Elimination (CSE), Constant Folding, Loop Invariant Code Motion (LICM), etc.

Compiler backend is the part of the compiler which is responsible for generating actual assembly or machine code for the architecture being targeted by the compiler. This stage involves target specific optimizations like Strength Reduction, Register Allocation, etc. This

stage takes in its input from the middle end of the compiler and generates the code which can finally run on the target machine.

As one can notice, the different stages of the compiler play quite distinct roles from each other and require different kinds of techniques and specialization to develop.

LLVM is a compiler infrastructure which makes heavy use of the marked separation between different stages of compilation [6]. When using LLVM, programming language designers are mainly concerned with developing a frontend which can take in the source code, and then emit an intermediate representation (IR) called the LLVM IR. Once the LLVM IR is generated, programming language developers rely on the LLVM backends existing for a large number of ISAs to generate assembly and machine code for their target architecture. The job of the LLVM backend is to take LLVM IR as input and generate assembly or machine code for the target ISA [3].

This project demonstrates the development and working of a LLVM backend for the MRISC32 ISA, for which no prior LLVM backend existed. To achieve this, LLVM offers 2 major frameworks named the SelectionDAG and the GlobalISel [6]. While SelectionDAG is the traditional instruction selection framework of LLVM, development of GlobalISel began to overcome the limitations posed by the architecture of SelectionDAG [6]. For this backend, GlobalISel is leveraged from the beginning.

While an early stage GCC backend for this target exists [4], the development of an LLVM backend is important as it enables integration with the broader LLVM infrastructure like Clang, MLIR, etc. Besides, the highly modular nature of LLVM allows researchers working on hardware and compilers to quickly prototype experiment ideas [3].

1.2 PROJECT DESCRIPTION

This project involves the end-to-end development of an LLVM backend for the MRISC32 ISA. The backend begins with TableGen-based register and instruction modeling, then integrates with the GlobalISel framework for instruction selection, handling of calling convention followed by assembly printing and ELF object file generation.

The key deliverable of this project include a functional LLVM backend that can compile LLVM IR targeting the MRISC32 architecture, generate MRISC32 assembly from LLVM IR input, produce ELF object files compatible with the existing MRISC32 toolchain, and correctly handle the MRISC32 calling convention. The project also includes a comparison of the generated code against the existing GCC backend for MRISC32.

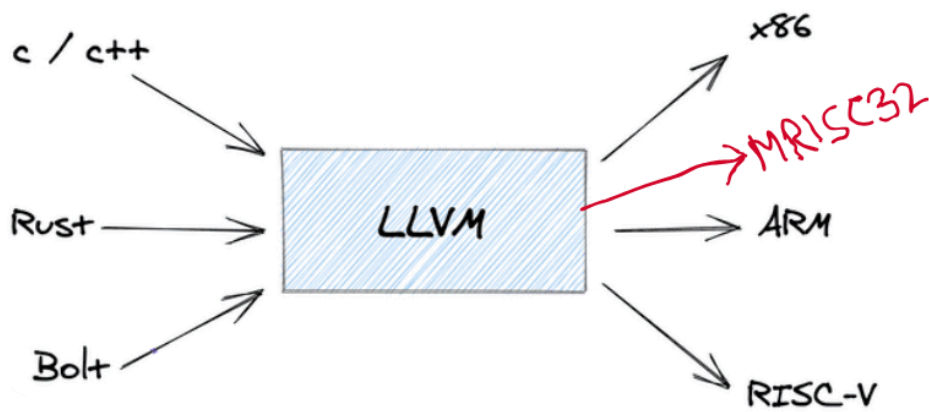


Figure 1.1. LLVM backed for MRISC32 ISA fits in the backend of the LLVM infrastructure along with backend for other architectures like x86, ARM, RISC-V, etc.

CHAPTER 2

LITERATURE REVIEW

2.1 LLVM COMPILER INFRASTRUCTURE

LLVM is a widely adopted open-source compiler infrastructure project that provides a collection of modular and reusable compiler and toolchain technologies. LLVM's design revolves around a well-defined intermediate representation (IR) that serves as the glue between language frontends and target-specific backends, enabling a high degree of code reuse across different languages and architectures.

The modular nature of LLVM enables researchers and developers to work on individual components of the compilation pipeline without affecting others. This has led to its widespread adoption in both academic research and industrial toolchains, with frontends such as Clang for C/C++, as well as backends for a large number of ISAs including x86, ARM, PowerPC, and RISC-V.

2.2 MRISC32 ISA

MRISC32 is an open and free instruction set architecture (ISA). It was designed by Marcus Geelnard as something similar to a RISC CPU.

It is a RISC style load-store vector architecture that is designed to be simple yet powerful and highly scalable. One of the main features of the instruction set architecture is its forward-looking vector functionality that not only integrates well with the rest of the ISA, but also enables implementations to freely select the level of hardware parallelism. This makes the vector functionality suitable for low-end systems like embedded microcontrollers as well as for high-performance computing (HPC). The advent and popularization of AI/ML applications and workloads also benefit greatly from an ISA design which does not fix the degree of parallelism.

2.3 INSTRUCTION SELECTION FRAMEWORKS

In compiler design theory, Instruction Selection is a particular step which involves converting a higher level IR to a more target specific IR. This is done as a process towards the lowering of IR to the target assembly or machine code. LLVM provides its own mechanism to perform instruction selection. Historically, the SelectionDAG was the dominant framework to perform this step. Another framework known as FastISel was also available. However, there were

challenges posed by both of these frameworks which led to the development of a new framework to perform this step called the GlobalISel.

2.3.1 SelectionDAG

SelectionDAG is the original and traditional instruction selection framework within LLVM. It works by first lowering LLVM IR into a directed acyclic graph (DAG) representation, where nodes represent operations and edges represent data dependencies. Pattern matching is then performed on this DAG to select target-specific instructions. While SelectionDAG has been proven effective for many architectures, it has certain architectural limitations, particularly around the handling of irregular operations and the difficulty of performing global optimizations due to its block-local scope.

2.3.2 GlobalISel

GlobalISel is a newer instruction selection framework introduced to address the limitations of SelectionDAG. Unlike SelectionDAG which operates on a per-basic-block basis, GlobalISel operates on the entire function at once, allowing for broader optimization opportunities. GlobalISel proceeds through four main stages: IRTranslator, Legalizer, RegBankSelect, and InstructionSelect. Bogner, Nandakumar, and Sanders presented the motivation and design of GlobalISel at the LLVM Developers Meeting, highlighting its advantages in terms of modularity, testability, and the ability to perform global analyses during instruction selection.

While an early-stage GCC backend exists for MRISC32, no prior LLVM backend was available for this architecture, thus motivating this work.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 TABLEGEN DOMAIN SPECIFIC LANGUAGE

Since the compiler backend is highly dependent on a large list of target-specific information such as the list of all available registers, instructions, their binary encodings, and instruction latency, a need for automating the generation of code required to record such information was felt. TableGen is a domain-specific language (DSL) within the LLVM project which is used for exactly this purpose [8]. This is done through defining TableGen classes, which act as a blueprint for instantiating individual records.

3.2 BACKEND DESIGN

3.2.1 Register Information

Since there are usually a large number of registers present in an ISA, each having potentially different properties, encoding, and width, the TableGen DSL is utilized to properly generate this record. In MRISC32, all general purpose registers can be used as the source register for an instruction, but register R0 cannot be used as the destination register. Therefore, a mechanism to model this constraint within the backend is required.

In this backend, this is achieved by creating two separate TableGen register classes named GPR and GPRDst. Register class GPR stores the list of all general purpose registers ranging from R0 to R31, whereas the register class GPRDst only stores those registers which are valid destination operands, i.e., registers R1 to R31.

```
33   def GPR      : MRISC32RegisterClass<[i32, f32], 32, 32, (add (sequence "r%u", 0, 31))>;
34   // Only consists of registers which can be used as destination register to store some values.
35   def GPRDst   : MRISC32RegisterClass<[i32, f32], 31, 31, (add (sequence "r%u", 1, 31))>;
```

Figure 3.1. Tablegen register classes GPR and GPRDst in MRISC32RegisterInfo.td

3.2.2 Instruction Information

Similar to registers, TableGen is utilized to record all the instructions and populate the information associated with each of them, including instruction encoding and instruction word width. Since there are 5 different formats of instructions in the MRISC32 ISA [5], this backend leverages the property of inheriting TableGen classes to define individual classes for each instruction format. These act as blueprints for defining individual instructions.

Individual instruction formats are defined as follows:

```

16 class FmtA<bits<10> opcode, dag outs, dag ins, string asmString>
17   : MRISC32Inst<outs, ins, asmString>
18 {
19     bits<5> regA;
20     bits<5> regB;
21     bits<1> V = {0b0};
22     bits<2> T = {0b00};
23
24     let Inst{31-28} = 0b0000;
25     let Inst{27-26} = opcode{9-8};
26     let Inst{25-21} = regA;
27     let Inst{20-16} = regB;
28     let Inst{15}    = V;
29     let Inst{14-9}  = opcode{7-2};
30     let Inst{8-7}   = T;
31     let Inst{6-2}   = 0b000000;
32     let Inst{1-0}   = opcode{1-0};
33 }

```

Figure 3.2. Instruction format A defined as “FmtA” in tablegen file [MRISC32InstrFormat.td](#)

Similarly individual tablegen classes are also defined for instruction format B, C, D and E.

These classes are then used to define individual instructions as follows:

```

8  /**
9   * ADD instruction
10  * Opcode: 22
11  * Format B and C
12  */
13 def ADD_B : FmtB<22, (outs GPRDst:$regA), (ins GPR:$regB, GPR:$regC), "add $regA, $regB, $regC">;
14 def ADD_C : FmtC<22, (outs GPRDst:$regA), (ins GPR:$regB, imm15:$imm), "add $regA, $regB, #imm">;

```

Figure 3.3. Two different formats of ADD instruction defined in [MRISC32InstrInfo.td](#) by inheriting the instruction formats B and C (FmtB and FmtC)

3.3 GlobalISel Pipeline

In LLVM, instruction selection is the process through which target-independent LLVM IR input is lowered and transformed into the limited set of instructions available for the target ISA, without breaking the overall meaning of the program being compiled. The GlobalISel pipeline involves four major stages as described below [7].

3.3.1 IRTranslator

The IRTranslator is responsible for mapping LLVM IR input to the Generic Machine IR (gMIR). This is the internal IR used in the GlobalISel framework. The LLVM backend for

MRISC32 implements the required target hooks to integrate with the GlobalISel pipeline at this stage.

3.3.2 Legalizer

The Legalizer is responsible for handling operations which are not natively supported by the target ISA. Due to the target-independent nature of LLVM IR, it is possible that certain LLVM IR instructions are not directly supported by the target architecture. In such a scenario, the Legalizer is responsible for transforming the unsupported operations into a sequence of supported operations.

To take a concrete example, adding two 64-bits numbers in the MRISC32 CPU is not natively possible because no instruction to handle such a case exists in ISA. To overcome this, this scenario is handled by the compiler. Whenever the compiler encounters two 64-bit numbers being added, the logic is handled to a custom defined function named *legalizeAdd()* defined below.

```
MRISC32LegalizerInfo::MRISC32LegalizerInfo(const MRISC32Subtarget &ST) :  
    LegalizerInfo()  
{  
    getLegacyLegalizerInfo().computeTables();  
  
    // Handle G_ADD with scalar 64 using custom legalization  
    getActionDefinitionsBuilder(TargetOpcode::G_ADD).customFor({LLT::scalar(64)});  
}
```

Figure 3.4. *MRISC32LegalizerInfo* class constructor containing logic to decide whether operation is natively handled or needs custom handling

```

48 bool MRISC32LegalizerInfo::legalizeAdd(MachineInstr &MI, MachineRegisterInfo &MRI,
49                                         MachineIRBuilder &MIRBuilder, GISELChangeObserver &Observer) const
50 {
51     Register Op1 = MI.getOperand(1).getReg();
52     Register Op2 = MI.getOperand(2).getReg();
53
54     LLT scalar32 = LLT::scalar(32);
55     LLT scalar1  = LLT::scalar(1);
56
57     auto unmergeOp1 = MIRBuilder.buildUnmerge(scalar32, Op1);
58     auto unmergeOp2 = MIRBuilder.buildUnmerge(scalar32, Op2);
59
60     Register lowBitsOp1 = unmergeOp1.getReg(0);
61     Register highBitsOp1 = unmergeOp1.getReg(1);
62
63     Register lowBitsOp2 = unmergeOp2.getReg(0);
64     Register highBitsOp2 = unmergeOp2.getReg(1);
65
66     Register lowBitsResult = MRI.createVirtualRegister(&MRISC32::GPRRegClass);
67     Register highBitsResult = MRI.createVirtualRegister(&MRISC32::GPRRegClass);
68     Register carry = MRI.createGenericVirtualRegister(scalar1);
69     Register carry32 = MRI.createGenericVirtualRegister(scalar32);
70
71     MIRBuilder.buildAdd(lowBitsResult, lowBitsOp1, lowBitsOp2);
72
73     // carry = (lowBitsResult < lowBitsOp1)
74     // NOTE: ICMP_ULT is not yet legalized by the MRISC32 backend
75     MIRBuilder.buildICmp(CmpInst::ICMP_ULT, carry, lowBitsResult, lowBitsOp1);
76
77     // extend carry to 32 bits
78     MIRBuilder.buildZExt(carry32, carry);
79
80     MIRBuilder.buildAdd(highBitsResult, highBitsOp1, highBitsOp2);
81     MIRBuilder.buildAdd(highBitsResult, highBitsResult, carry32);
82
83     // combine into a 64-bit result
84     Register ResultReg = MI.getOperand(0).getReg();
85     MIRBuilder.buildMergeValues(ResultReg, {lowBitsResult, highBitsResult});
86
87     MI.eraseFromParent();
88     return true;
89 }

```

Figure 3.5. Custom logic to handle 64 bit addition written in C++

3.3.3 RegBankSelect

RegBankSelect is responsible for assigning the virtual registers in which values are stored to the register banks available in the CPU. This stage ensures that all virtual registers are associated with an appropriate register bank before the final instruction selection pass.

Code example:

3.3.4 InstructionSelect

InstructionSelect is the stage during which generic Machine IR (gMIR) obtained from the previous stages of GlobalISel is mapped to the target-specific Machine IR (MIR). In

GlobalISel, one way of achieving this is by first converting the gMIR to equivalent SDNode representations, and then pattern matching is performed to generate correct MIR. Thanks to the pattern matching performed, when the incoming LLVM IR instruction is of the form - %a = add i32 %b, %c, it is matched with the register-register format of the MRISC32 ADD instruction, whereas when the LLVM IR instruction is - %a = add i32 %b, 8, the register-immediate format of the ADD instruction is generated.

Code example:

```

16  def : Pat<(add GPR:$regB, GPR:$regC),
17  |      |      |      (ADD_B GPR:$regB, GPR:$regC)>;
18  def : Pat<(add GPR:$regB, imm:$imm),
19  |      |      |      (ADD_C GPR:$regB, imm:$imm)>;
20
21  def : GINodeEquiv<G_ADD, add>;

```

Figure 3.6. Tablegen instruction to define pattern matching

3.3.5 Calling Convention Handling

Calling convention refers to the architecture or operating system specific guidelines which govern how function arguments are passed, how results are returned out of a function and how stack is managed during runtime. Different architectures define different calling conventions depending on the ISA specific properties such as number of available registers and so on. These are usually part of ISA specification documents.

To follow the calling convention defined by the MRISC32 ISA, the first 8 function arguments must be passed through registers ranging from R1 to R8. In the LLVM backend, this is materialized within the `lowerFormalArguments()` function in `MRISC32CallLowering.cpp`. The code generates MIR instructions to copy the first 8 arguments using physical registers. Once the control reaches the end of a function block, the value which has to be returned by the current function is stored in register R1.

```
40 bool MRISC32CallLowering::lowerFormalArguments(  
41     MachineIRBuilder &MIRBuilder, const Function &F,  
42     ArrayRef<ArrayRef<Register>> VRegs, FunctionLoweringInfo &FLI) const {  
43  
44     // The first arguments to a function are passed in registers R1 to R8.  
45     static const MCPhysReg ArgRegs[] = {MRISC32::r1, MRISC32::r2, MRISC32::r3, MRISC32::r4,  
46         |   |   |   |   |   |   |   |   | MRISC32::r5, MRISC32::r6, MRISC32::r7, MRISC32::r8};  
47  
48     // Map each incoming VReg to a physical register  
49     for (unsigned i = 0; i < VRegs.size(); ++i) {  
50         if (i >= 8) return false;           // First 8 args are passed through registers  
51  
52         Register VReg = VRegs[i][0];  
53         MCPhysReg PReg = ArgRegs[i];  
54  
55         // Add the physical register to the function live-ins  
56         MIRBuilder.getMBB().addLiveIn(PReg);  
57  
58         // Create the definition for the virtual register  
59         MIRBuilder.buildCopy(VReg, Register(PReg));  
60  
61         // TODO: Define logic to push greater than 8th argument to the stack  
62     }  
  
63     return true;  
64 }  
65 }
```

Figure 3.7. *lowerFormalArguments()* is responsible for passing the first 8 function parameters to register R1 to R8

The MRISC32 ISA also defines that the return values are stored in register R1. This is implemented using the function *lowerReturn()* in the MRISC32CallLowering.cpp file.

```

18 MRISC32CallLowering::MRISC32CallLowering(const MRISC32TargetLowering &TLI)
19     : CallLowering(&TLI) {}
20
21 bool MRISC32CallLowering::lowerReturn(MachineIRBuilder &MIRBuilder, const Value *Val,
22     ArrayRef<Register> VRegs, FunctionLoweringInfo &FLI,
23     Register SwiftErrorVReg) const {
24
25     // If there is a return value (VRegs is not empty), move it to R1
26     if (!VRegs.empty()) {
27         MIRBuilder.buildCopy(MRISC32::r1, VRegs[0]);
28     }
29
30     MIRBuilder.buildInstr(MRISC32::RET).addUse(MRISC32::r1);
31     return true;
32 }

```

Figure 3.8. *lowerReturn()* is responsible for moving return value to register R1

3.3.6 Function Prologue and Epilogue Generation

1. Function Prologue

Function prologue is the few lines of code at the beginning of every function. These few lines of instructions are necessary to prepare the stack and registers for use within a function.

In the LLVM backend for MRISC32, the function prologue is generated by a function named *emitPrologue()*. This function defines the set of instructions which has to be generated as part of the Prologue section for the functions in the target ISA.

The MRISC32 assembly generated by this backend has the prologue section which contains the following set of instructions.

- Adding stack size as offset to current value of stack pointer
 - `add sp, sp, #-24`
- Storing frame pointer register to stack
 - `stw fp, [sp, #20]`
- Saving top of current stack frame in frame pointer
 - `add fp, sp, #24`

2. Function Epilogue

Function epilogue is the few lines of code at the end of every function. These few lines of instructions are used to restore the state of the stack and registers to the state before the function call.

In the MRISC32 backend, the function epilogue is generated using a function named *emitEpilogue()*. This function defines the set of instructions which has to be generated as part of the epilogue for the functions in the target ISA.

In the MRISC32 assembly generated by this backend, the epilogue section contains the following set of instructions.

- Restore the old frame pointer
 - `ldw fp, [sp, #20]`
- Deallocate the stack frame
 - `add sp, sp, #24`
- Jump out from current function to the caller function
 - `ret`

```

33 void MRISC32FrameLowering::emitPrologue(MachineFunction &MF, MachineBasicBlock &MBB) const {
34     MachineBasicBlock::iterator MBBI = MBB.begin();
35     MachineFrameInfo &MFI = MF.getFrameInfo();
36
37     uint64_t stackSize = alignTo(MFI.getStackSize(), getStackAlign());
38     MFI.setStackSize(stackSize);
39
40     if(stackSize == 0) {
41         return;
42     }
43
44     DebugLoc DL;
45
46     Register framePointer = MRISC32::r28;
47     Register stackPointer = MRISC32::r29;
48
49     BuildMI(MBB, MBBI, DL, TII.get(MRISC32::ADD_C))
50         .addReg(stackPointer, RegState::Kill)
51         .addReg(stackPointer, RegState::Kill)
52         .addImm(-stackSize)
53         .setMIFlag(MachineInstr::FrameSetup);
54
55     BuildMI(MBB, MBBI, DL, TII.get(MRISC32::STW_C))
56         .addReg(framePointer, RegState::Kill)
57         .addReg(stackPointer, RegState::Kill)
58         .addImm(stackSize - 4);
59
60     BuildMI(MBB, MBBI, DL, TII.get(MRISC32::ADD_C))
61         .addReg(framePointer)
62         .addReg(stackPointer)
63         .addImm(stackSize);
64 }

```

Figure 3.9. *emitPrologue()* is responsible for generation prologue for every function

```

66 void MRISC32FrameLowering::emitEpilogue(MachineFunction &MF, MachineBasicBlock &MBB) const {
67     MachineBasicBlock::iterator MBBI = MBB.getFirstTerminator();
68     MachineFrameInfo &MFI = MF.getFrameInfo();
69
70     uint64_t stackSize = MFI.getStackSize();
71
72     if(stackSize == 0) {
73         return;
74     }
75
76     DebugLoc DL;
77
78     Register framePointer = MRISC32::r28;
79     Register stackPointer = MRISC32::r29;
80
81     BuildMI(MBB, MBBI, DL, TII.get(MRISC32::LDW_C))
82         .addReg(framePointer, RegState::Kill)
83         .addReg(stackPointer, RegState::Kill)
84         .addImm(stackSize - 4);
85
86     BuildMI(MBB, MBBI, DL, TII.get(MRISC32::ADD_C))
87         .addReg(stackPointer)
88         .addReg(stackPointer)
89         .addImm(stackSize);
90
91 }

```

Figure 3.10. *emitEpilogue()* is responsible for generating epilogue at the end of every function

3.3.7 Assembly Generation

Once the instruction selection process is completed, target-independent LLVM IR is transformed into target-specific MIR. However, further processing is required to generate human readable assembly from this. This is done by converting MIR to the MC layer and then printing assembly from it. This stage includes formatting registers and immediate operands. The *MRISC32InstPrinter* class is utilized for this purpose, printing the instruction name and operands along with proper formatting.

```
13 class MRISC32InstPrinter : public MCInstPrinter {
14 public:
15     MRISC32InstPrinter(const MCAsmInfo &MAI, const MCInstrInfo &MII,
16                       const MCRegisterInfo &MRI)
17         : MCInstPrinter(MAI, MII, MRI) {}
18
19     void printInst(const MCInst *MI, uint64_t Address, StringRef Annot,
20                   const MCSubtargetInfo &STI, raw_ostream &O) override;
21
22     void printOperand(const MCInst *MI, unsigned OpNo, raw_ostream &O);
23     //void printMemOperand(const MCInst *MI, int OpNo, raw_ostream &O);
24     //void printImm64Operand(const MCInst *MI, unsigned OpNo, raw_ostream &O);
25     //void printBrTargetOperand(const MCInst *MI, unsigned OpNo, raw_ostream &O);
26
27     // Autogenerated by tblgen.
28     std::pair<const char *, uint64_t>
29     getMnemonic(const MCInst &MI) const override;
30     void printInstruction(const MCInst *MI, uint64_t Address, raw_ostream &O);
31     static const char *getRegisterName(MCRegister Reg, unsigned AltIdx = 0);
32 };
33 }
```

Figure 3.11. *MRISC32InstPrinter* class utilized for defining functions to print instructions and operands in proper format

3.3.8 ELF Generation

Apart from generating assembly, the LLVM framework also provides hooks to directly generate object files in formats like ELF and MachO. This stage is responsible for appropriately writing the ELF header and generating the machine code in ELF format. In the MRISC32 ISA, several instructions are defined as aliases for a number of fundamental instructions. While the assembly output is not expected to expand such aliases since they improve readability, the generated object files must properly expand aliases and encode the correct sequence of fundamental instructions. The *encodeInstruction()* function in

MRISC32MCCodeEmitter.cpp checks whether the incoming instruction is an alias or a fundamental instruction, and based on this, the correct opcode is generated and written to the object file.

```
99 void MRISC32MCCodeEmitter::encodeInstruction(const MCInst &MI,  
100                                             SmallVectorImpl<char> &CB,  
101                                             SmallVectorImpl<MCFixup> &Fixups,  
102                                             const MCSubtargetInfo &STI) const {  
103     if(MI.getOpcode() == MRISC32::RET)  
104     {  
105         uint32_t expInstrEnc = 0xc3c00000; // j lr, #0  
106         support::endian::write<uint32_t>(CB, expInstrEnc, llvm::endianness::little);  
107         return;  
108     }  
109  
110     // Get instruction encoding and emit it  
111     uint64_t Encoding = getBinaryCodeForInstr(MI, Fixups, STI);  
112     assert(((Encoding & 0xffffffff00000000) == 0) &&  
113           "Only the first 32-bit should be set");  
114     support::endian::write<uint32_t>(CB, Encoding, llvm::endianness::little);  
115 }
```

Figure 3.12. *encodeInstruction()* in *MRISC32MCCodeEmitter.cpp* used to generate the correct opcode

CHAPTER 4

RESULTS AND DISCUSSION

To evaluate the correctness and performance of the developed LLVM backend, a comparison is conducted between the assembly code generated by the LLVM backend and the assembly code generated by the existing GCC backend for MRISC32. A simple C function performing a sequence of arithmetic operations was used as the test case.

4.1 TEST FUNCTION

The test function used for comparison performs three operations: addition of two integer arguments, subtraction of a constant, and integer division. The function takes two integer arguments and returns the result of these operations.

```
int func(int a, int b) {  
    int c = a + b;  
    int d = c - 9;  
    int e = d / 7;  
    return e;  
}
```

Figure 4.1. Test program written in C consisting of sequence of arithmetic operations

4.2 GCC BACKEND OUTPUT

The GCC backend for MRISC32 was invoked using the *mrisc32-elf-gcc -S func.c -o func.s* command. The resulting assembly uses frame pointer (fp) based addressing for all local variables, with explicit store and load operations around each arithmetic instruction.

```

func:
add sp, sp, #-24
stw fp, [sp, #20]
add fp, sp, #24
stw r1, [fp, #-20]
stw r2, [fp, #-24]
ldw r2, [fp, #-20]
ldw r1, [fp, #-24]
add r1, r2, r1
stw r1, [fp, #-8]
ldw r1, [fp, #-8]
add r1, r1, #-9
stw r1, [fp, #-12]
ldw r1, [fp, #-12]
div r1, r1, #7
stw r1, [fp, #-16]
ldw r1, [fp, #-16]
ldw fp, [sp, #20]
add sp, sp, #24
ret

```

Figure 4.2. MRISC32 assembly generated by the existing GCC backend

4.3 LLVM BACKEND OUTPUT

To use the LLVM backend, LLVM IR was first generated from the C source using the Clang compiler with the *-emit-llvm* flag. The LLVM IR thus generated, was then fed into the MRISC32 LLVM backend to produce MRISC32 assembly. The resulting assembly uses stack-based addressing for local variables, with a structure similar to the GCC backend output.

LLVM IR

```
define dso_local i32 @func(i32 noundef %0, i32 noundef %1) #0 {  
    %3 = alloca i32, align 4  
    %4 = alloca i32, align 4  
    %5 = alloca i32, align 4  
    %6 = alloca i32, align 4  
    %7 = alloca i32, align 4  
    store i32 %0, ptr %3, align 4  
    store i32 %1, ptr %4, align 4  
    %8 = load i32, ptr %3, align 4  
    %9 = load i32, ptr %4, align 4  
    %10 = add nsw i32 %8, %9  
    store i32 %10, ptr %5, align 4  
    %11 = load i32, ptr %5, align 4  
    %12 = sub nsw i32 %11, 9  
    store i32 %12, ptr %6, align 4  
    %13 = load i32, ptr %6, align 4  
    %14 = sdiv i32 %13, 7  
    store i32 %14, ptr %7, align 4  
    %15 = load i32, ptr %7, align 4  
    ret i32 %15  
}
```

Figure 4.3. LLVM IR generated from clang using *-emit-llvm* flag

MRISC32 Assembly

```
func:  
    add sp, sp, #-24  
    stw fp, [sp, #20]  
    add fp, sp, #24  
    ldi r3, #9  
    stw r1, [fp, #-8]  
    stw r2, [fp, #-12]  
    ldw r1, [fp, #-8]  
    ldw r2, [fp, #-12]  
    add r1, r1, r2  
    stw r1, [fp, #-16]  
    ldw r1, [fp, #-16]  
    sub r1, r1, r3  
    stw r1, [fp, #-20]  
    ldw r1, [fp, #-20]  
    div r1, r1, #7  
    stw r1, [fp, #-24]  
    ldw r1, [fp, #-24]  
    ldw fp, [sp, #20]  
    add sp, sp, #24  
    ret
```

Figure 4.4. MRISC32 assembly generated from LLVM IR using LLVM backend for MRISC32

4.4 COMPARISON AND ANALYSIS

The assembly code generated by both the LLVM and the GCC backends are semantically equivalent and produce the same results for the supported instructions. Both backends

correctly implement the MRISC32 calling convention, passing function arguments through registers R1 and R2, and returning the result in register R1.

Minor variations exist between the two outputs. Notably, the LLVM backend, in certain scenarios, first loads an immediate value into a register using the `ldi` instruction and then performs a register-register operation, whereas the GCC backend directly uses the immediate to perform a register-immediate operation. For example, the constant 9 used for subtraction is handled differently: the LLVM backend loads it into register r3 first (`ldi r3, #9`), whereas the GCC backend directly uses `add r1, r1, #-9`. Despite this minor stylistic difference, the final computational results are identical.

The successful generation of semantically correct and functionally equivalent assembly code by the LLVM backend validates the correct implementation of the GlobalISel pipeline, calling convention handling, and assembly printing components of this project.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

This project presented the design and implementation of a LLVM backend for the MRISC32 ISA using the GlobalISel instruction selection framework. The backend successfully generates MRISC32 assembly from input LLVM IR and produces ELF object files compatible with the existing MRISC32 toolchain.

Key components implemented include register modeling using TableGen register classes GPR and GPRDst, instruction definitions for all five MRISC32 instruction formats using inheritable TableGen classes, and the complete GlobalISel pipeline comprising IRTranslator, Legalizer, RegBankSelect, and InstructionSelect. The calling convention is properly handled by passing the first eight function arguments through registers R1 to R8 and returning values via register R1. Assembly generation with proper register and immediate formatting is supported through the MRISC32InstPrinter class, and ELF object file generation with correct alias expansion is implemented via MRISC32MCCodeEmitter.

Comparison with the existing GCC backend for MRISC32 demonstrated that the LLVM backend generates semantically equivalent assembly for the supported Base module instructions. The current implementation establishes a solid and functional foundation for the MRISC32 architecture within the LLVM ecosystem.

5.2 FUTURE SCOPE

The MRISC32 ISA consists of a mandatory Base Module plus a number of optional modules. These optional modules include the Vector operation module (VM), Packed operation module (PM), Floating-point module (FM), and the Saturating and halving arithmetic module (SM). The current version of the LLVM backend for the MRISC32 ISA only targets the Base module as a starting point.

In future, this project can be extended to provide support for vector and packed operations, which could potentially benefit AI/ML and embedded workloads. Support for floating-point operations through the FM module can be integrated to enable a broader class of scientific and numerical applications. Additionally, the LLVM toolchain for MRISC32 currently relies on the GNU-based linker and standard library (newlib); this project can be extended to leverage the LLVM infrastructure and create its own linker and standard library, enabling a

fully self-contained LLVM-based toolchain for MRISC32. Optimization passes specific to the MRISC32 architecture can also be developed to improve the quality of the generated code.

REFERENCES

- [1] W. Harrison and B. Adrangi, "The role of programming language in estimating software development costs," *J. Manage. Inf. Syst.*, vol. 3, no. 3, pp. 101-110, 1986.
- [2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2007.
- [3] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis and transformation," in *Proc. Int. Symp. Code Gener. Optim. (CGO)*, Mar. 2004, pp. 75-86.
- [4] MRISC32 Project, "GCC for MRISC32," GitLab repository. [Online]. Available: <https://gitlab.com/mrisc32/gcc-mrisc32>. [Accessed: Feb. 28, 2026].
- [5] MRISC32 Project, "MRISC32 instruction set manual," GitLab. [Online]. Available: <https://gitlab.com/mrisc32/mrisc32/-/releases/permalink/latest/downloads/mrisc32-instruction-set-manual.pdf>. [Accessed: Feb. 28, 2026].
- [6] C. Lattner, "LLVM," in *The Architecture of Open Source Applications*, vol. 1. [Online]. Available: <https://aosabook.org/en/v1/llvm.html>. [Accessed: Feb. 28, 2026].
- [7] A. Bogner, A. Nandakumar, and D. Sanders, "Head first into GlobalISel," in *Proc. LLVM Developers' Meeting*, Oct. 2017.
- [8] LLVM Project, "TableGen documentation," LLVM Documentation. [Online]. Available: <https://llvm.org/docs/TableGen/index.html>. [Accessed: Feb. 28, 2026]

Development of LLVM Backend for MRISC32 ISA using GlobalISel

Shobhit Sinha

Dept. of CSE (Artificial Intelligence
and Machine Learning)
KIET Group of Institutions
Ghaziabad, India
Shobhit32t@gmail.com

Pratibha Singh

Dept. of CSE (Artificial Intelligence
and Machine Learning)
KIET Group of Institutions
Ghaziabad, India
pratibha.singh@kiet.edu

Abstract—This paper demonstrates the development and working of a LLVM backend for the MRISC32 ISA, utilizing the GlobalISel framework.

Index Terms—LLVM, Compiler Backend, MRISC32, GlobalISel, Instruction selection

I. INTRODUCTION

Instruction Set Architecture (ISA) is the interface available to the programmer through which a processor is programmed. Since modern ISAs and processors are highly complex in nature, it is impractical to consistently write highly optimized programs for even trivial tasks directly in assembly or machine code [1]. Due to this limitation, the development of any modern ISA infrastructure is incomplete without the development of a compiler toolchain to support it. Modern compiler infrastructure frameworks like LLVM greatly reduce the time and effort required to create such a toolchain.

The process of compiler construction can be mainly broken down into 3 major parts. These are the compiler frontend, middlend and the backend [2]. LLVM is a compiler infrastructure which makes heavy use of the marked separation between different stages of compilation [6]. When using LLVM, programming language designers are mainly concerned with developing a frontend which can take in the source code, and then emit an intermediate representation (IR) called the LLVM IR. Once the LLVM IR is generated, programming language developers rely on the LLVM backends existing for a large number of ISAs to generate assembly and machine code for their target architecture. The job of the LLVM backend is to take LLVM IR as input and generate assembly or machine code for the target ISA [3].

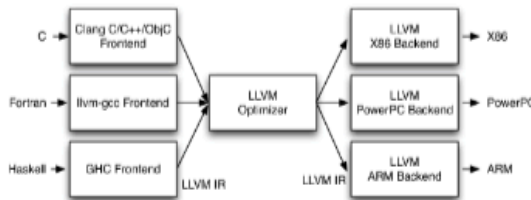


Fig. 1: LLVM's Implementation of the Three-Phase Design [6]

This paper demonstrates the development and working of a LLVM backend for the MRISC32 ISA, for which no prior LLVM backend existed. To achieve this, LLVM offers 2 major frameworks named the SelectionDAG and the GlobalISel [6].

While SelectionDAG is the traditional instruction selection framework of LLVM, development of GlobalISel began to overcome the limitations posed by the architecture of SelectionDAG [6]. For this backend, GlobalISel is leveraged from the beginning.

While an early stage GCC backend for this target exists [4], the development of an LLVM backend is important as it enables integration with the broader LLVM infrastructure like Clang, MLIR, etc. Besides, the highly modular nature of LLVM allows researchers working on hardware and compilers to quickly prototype experiment ideas [3].

II. MRISC32 ISA

MRISC32 is an open-source RISC-style load-store vector architecture that is designed with scalability in mind [5]. There are 32 registers available, each of which are 32 bits wide, and are numbered from R0 to R31. Similar to other RISC style ISAs like MIPS and RISC-V, the R0 register in MRISC32 is also hardwired with value 0. Writing to this register has no effect [5].

All instructions of MRISC32 ISA are 32 bits wide, and are encoded in 5 different formats - A, B, C, D and E, and differ from each other in the number and type of operands used [5].

The calling convention defined by the MRISC32 ISA states that the first 8 arguments to a function are passed through registers R1-R8. If a function has additional arguments, they must be passed through stack [5].

III. TABLEGEN DOMAIN SPECIFIC LANGUAGE

Since the compiler backend is highly dependent on a large list of target specific information such as list of all available registers, instructions, their binary encodings, instruction latency, etc, a need for automating the generation of code required to record such information was felt. Tablegen is a domain-specific language (DSL) within the LLVM project which is used for exactly this purpose [8]. This is done through defining tablegen classes, which act as a blueprint for instantiating individual records.

IV. BACKEND DESIGN

A. Register information

Since usually there is a large number of registers present in an ISA, each having potentially different properties, encoding, and width, the tablegen DSL is utilised to properly generate this record.

In MRISC32, all general purpose registers can be used as the source register for an instruction but register R0 cannot

PROOF OF SUBMISSION

International Conference on Computing, Communication and Artificial Intelligence (IC3AI) : Submission (30) has been created. [Inbox x](#)



Microsoft CMT <noreply@msr-cmt.org>
to me ▾

Wed 15 Apr, 04:37 ☆ 😊 ↩ ⋮

Hello,

The following submission has been created.

Track Name: Advanced Computing and Application

Paper ID: 30

Paper Title: Development of LLVM Backend for MRISC32 ISA using GlobalISel

Abstract:

This paper demonstrates the development and working of a LLVM backend for the MRISC32 ISA, utilizing the GlobalISel framework.

Created on: Tue, 14 Apr 2026 23:07:00 GMT

Last Modified: Tue, 14 Apr 2026 23:07:00 GMT

Authors:
- shobhit32t@gmail.com (Primary)

Secondary Subject Areas: Not Entered

Submission Files:
LLVM Backend for MRISC32 ISA Using GlobalISel.pdf (121 Kb, Tue, 14 Apr 2026 23:03:48 GMT)

Submission Questions Response:
1. Author Consent
Agreement accepted

Thanks,
CMT team.

PLAGIARISM REPORT: AI SIMILARITY



Page 1 of 29 - Cover Page

Submission ID trn:oid::3618:137428070

Priya Singh

KIET_BTech_Report_LLVM_MRISC32

 Pratibha Singh uploaded Assignment

Document Details

Submission ID

trn:oid::3618:137428070

Submission Date

May 2, 2026, 6:32 PM GMT+5:30

Download Date

May 2, 2026, 6:37 PM GMT+5:30

File Name

KIET_BTech_Report_LLVM_MRISC32.docx

File Size

3.6 MB

27 Pages

4,211 Words

24,168 Characters



Page 1 of 29 - Cover Page

Submission ID trn:oid::3618:137428070

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



PLAGIARISM REPORT: OVERALL SIMILARITY



Page 1 of 32 - Cover Page

Submission ID trn:oid::3618:137428070

Priya Singh

KIET_BTech_Report_LLVM_MRISC32

Pratibha Singh uploaded Assignment

Document Details

Submission ID

trn:oid::3618:137428070

Submission Date

May 2, 2026, 6:32 PM GMT+5:30

Download Date

May 2, 2026, 6:37 PM GMT+5:30

File Name

KIET_BTech_Report_LLVM_MRISC32.docx

File Size

3.6 MB

27 Pages

4,211 Words

24,168 Characters



Page 1 of 32 - Cover Page

Submission ID trn:oid::3618:137428070

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- 34 Not Cited or Quoted 9%**
Matches with neither in-text citation nor quotation marks
- 1 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 6% Internet sources
- 3% Publications
- 6% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **34 Not Cited or Quoted 9%**
Matches with neither in-text citation nor quotation marks
- **1 Missing Quotations 0%**
Matches that are still very similar to source material
- **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 6% Internet sources
- 3% Publications
- 6% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	en.everybodywiki.com	<1%
2	Student papers	The Indian Institute Of Management And Engineering Society on 2026-05-02	<1%
3	Student papers	University at Buffalo on 2023-10-13	<1%
4	Internet	a4cp.org	<1%
5	Student papers	Indian Institute of Technology IIT Palakkad on 2023-05-16	<1%
6	Student papers	Imperial College of Science, Technology and Medicine on 2026-01-22	<1%
7	Publication	Saumya Debray. "Making compiler design relevant for students who will (most lik...	<1%
8	Internet	dspace.iti.ac.in	<1%
9	Internet	idoc.pub	<1%
10	Publication	Behrooz Sangchoolie, Fatemeh Ayatollahi, Roger Johansson, Johan Karlsson. "A Co...	<1%

11	Internet	grotrian.nsu.ru	<1%
12	Internet	tudr.thapar.edu:8080	<1%
13	Student papers	University at Buffalo on 2023-10-17	<1%
14	Internet	www.ece.lsu.edu	<1%
15	Student papers	King's College on 2019-04-08	<1%
16	Student papers	University Politehnica of Bucharest on 2025-06-26	<1%
17	Student papers	University of Edinburgh on 2011-04-26	<1%
18	Student papers	Mapua University on 2025-10-02	<1%
19	Student papers	University College London on 2020-09-29	<1%
20	Internet	dspace.cvut.cz	<1%
21	Internet	repository.charlotte.edu	<1%
22	Student papers	Swinburne University of Technology on 2025-06-01	<1%
23	Student papers	University of Edinburgh on 2011-09-16	<1%
24	Student papers	University of Limerick on 2024-08-15	<1%